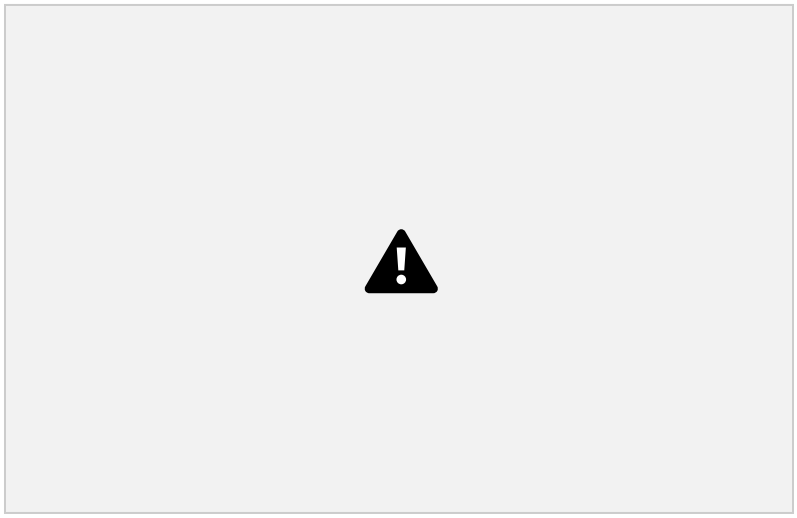
Diabetes Classification using Ensemble Learning and Random Forests



Introduction

In this assignment you will use the **classic UCI Pima Indians Diabetes dataset,** inspect its structure, handle implicit missing data, construct preprocessing pipelines using Scikit-Learn, and implement ensemble models, including **Voting Classifiers**, **Bagging with Out-of-Bag evaluation**, **Random Forests**, and **AdaBoost,** to predict the presence of **diabetes**.

Setup

- Download the UCI Pima Indians Diabetes dataset from the **UCI Machine Learning Repository**.
- Use Google Colab as your cloud-based runtime to **execute all notebooks.**
- Save your completed notebook under your **HW5** folder as **diabetes.ipynb**.

Dataset Overview

The dataset contains medical diagnostic measurements and patient data used to model the onset of diabetes.

- **Target Variable (outcome)**: A binary indicator of **diabetes** presence (0 for non-diabetic, 1 for diabetic). This serves as the primary outcome variable for predictive classification modeling.

- **Physiological Features**: Features such as **pregnancies**, **glucose** (plasma glucose concentration), **blood pressure** (diastolic blood pressure), **skin thickness** (triceps skin fold thickness), **insulin** (2-hour serum insulin), **bmi** (body mass index), **diabetes pedigree** function, and **age** quantify metabolic health and risk factors.
- **Implicit Missing Values Note:** Proper **identification** and **imputation** of **implicit** missing values are required before modeling.

Technical Clarification

Before implementing the classification models, note the fundamental architectural distinction across ensemble methods:

- **Parallel Ensembles (Voting, Bagging, Random Forests)**:
    - These methods train multiple base estimators independently or in parallel.
    - **Bagging** and **Random Forests** reduce model variance through **bootstrap sampling** and **random feature subsetting**, while **Voting Classifiers** combine distinct algorithm types via **majority rule** or **probability averaging**.

- **Sequential Ensembles (Boosting / AdaBoost)**:
    - This models trains predictors **sequentially**.
    - Each subsequent model focuses heavily on the **instances misclassified or underfitted** by its predecessor through adaptive sample weighting and **decision stumps**.

Tasks

- Create a Jupyter notebook named **diabetes.ipynb.**
- Your notebook code cells must directly correspond to each sequential section outlined below, and you must clearly mark each task using a dedicated Markdown text cell placed immediately before the corresponding code cell.
- **Each text cell must contain an extensive and well-formatted explanation of what the following code cell (Task) does**.
- **You will not receive full credit if the above-explained text cells are not included in your report**.

Perform the following tasks:

- **Task 1**: Load the dataset into a Pandas DataFrame from the source repository or course mirror, assigning appropriate column names.
- **Task 2**: Use structural inspection methods (head(), info(), describe()) to analyze feature distributions and identify non-standard placeholder values.
- **Task 3**: Quantify and handle implicit missing values using **SimpleImputer**.
- **Task 4**: Build a Scikit-Learn **ColumnTransformer** and **Pipeline** that applies appropriate **scaling**.
- **Task 5**: Train both a **Hard Voting Classifier** and a **Soft Voting Classifier** combining base models (e.g., **Logistic Regression**, **Support Vector Classifier**, and a **Decision Tree**), then evaluate their **test accuracy** and **classification reports**. Use must use appropriate **plots** to show the results.
- **Task 6**: Train a **Bagging Classifier using decision tree base estimators** with **bootstrap sampling** enabled and **out-of-bag evaluation**, and compare the OOB score against held-out test accuracy. **Plot** the accuracy results in one graph.
- **Task 7**: Train a **Random Forest Classifier** configured with **feature subsetting**, evaluate its test performance, and generate a horizontal bar plot visualizing feature importances (Mean Decrease in Impurity).
- **Task 8**: **Train an AdaBoost Classifier** utilizing **decision stumps** and sequential sample weighting, and evaluate its training versus test performance to observe potential boosting generalization trends. **Plot** the results.

Analysis

You must answer each of the following questions clearly, thoroughly, and in **full detail within a separate analysis document** or markdown report inside your **HW5** folder. **Shallow and brief answers will not receive full credit.**

1. Explain how implicit missing values impact medical datasets, and justify your choice of imputation strategy when handling these zeros compared to outright row deletion.
2. Contrast how hard voting aggregates predictions versus soft voting. Why do well-calibrated probability estimates matter when implementing soft voting across heterogeneous models?

3. What is the role of out-of-bag evaluation in Bagging classifiers, and why does an OOB score serve as an efficient alternative to traditional cross-validation splits during ensemble training?
4. How does introducing random feature subsetting in Random Forests differ from standard Bagging, and how does this feature diversity impact model variance and generalization?
5. Discuss the mechanics of sequential sample weighting in AdaBoost. Based on your training versus test accuracy results, did AdaBoost exhibit signs of overfitting, and how can **hyperparameter adjustments** mitigate this?

Submission Guidelines

- Submit the link to your GitHub repository showing that your executed notebook (**diabetes.ipynb**) has been successfully saved under your **HW5** folder. The **dataset** must also be included in the folder.
- The **analysis** section must be completed as a **separate Jupyter notebook** by itself where you answer the questions, and this analysis notebook must also be included inside your **HW5** folder.
- If your repo is private, you need to share it with me (**bforoura@sju.edu**).
- **Late submissions will incur a penalty of 10% per day as outlined in the course syllabus.**